MLR 570 · THE PROJECT GUIDE (THE ONE DOCUMENT)

Priority-Constrained Descent for Continual Learning

What we are testing, how we test it, what every number means, and where we stand

Faris Nasser · Salah Moussa

Instructor: Dr. Mohamed AlHajri · TA: Dara Varam · American University of Sharjah · Fall 2026

Status as of 7 November 2026 · **paper deadline: 19 November 2026**

This replaces every earlier PDF (the old guide, the brief, the iPad notes, the explainers). If something here disagrees with an older file, this wins. The paper draft lives separately in `paper/draft/`.

How to read it. red = key definitions and results blue = why it matters teal = mental model

amber = careful: limits, caveats, what we must not claim gray = worked example

Every experimental number names the experiment it came from (F-0xx). Every figure is drawn by `analysis/guide_figures.py` straight from the run files, and `paper/guide/figures/MANIFEST.md` lists each plotted number with its run IDs. Nobody typed a number into this document by hand without its ID next to it.

If you only have 20 minutes: Chapter 1 (one page), Chapter 5 (how to read a result), Chapter 8 (what we found) and Chapter 9 (the six questions you must be able to answer). Chapters 3 and 4 are the math; read them once, slowly.

Every result in this guide is exploration-stage

All numbers so far use the 10% validation split, one untuned learning rate (0.1) for every method, and 1–3 seeds. They choose which configurations go into the final runs. The final, reportable numbers (3 seeds × 3 task orders, test split, tuned learning rate) are run between 10 and 15 November.

Contents

1 The Project in One Page 5

2 The Setup: Data, Models and the Words We Use 7

2.1 One continual-learning run, as a picture 7

2.2 The two datasets 7

2.3 The two ways to remember 8

2.4 Dictionary: every word in this guide 9

3 Background You Need 10

3.1 The dot product tells you whether two directions agree 10

3.2 Gradient descent, seen through the first-order picture 10

3.3 Continual learning: the setting 11

3.4 How continual learning is measured: the accuracy matrix 11

3.5 Why forgetting happens, in one line 12

3.6 The existing fixes 13

4 How PCD Works 14

4.1 The idea: anchor to the primary, distort the minimum 14

4.2 The PCD problem 14

4.3 The picture, and the closed form for one constraint 15

4.4 Normalization: why τ means the same thing for every loss 16

4.5 The rescale step 17

4.6 The threshold: when protecting the old task stalls the new one 17

4.7 Many old tasks: the general solution 18

4.8 The algorithm in plain steps 18

4.9 What PCD promises, and what it does not 19

5 Reading a Result 21

5.1 The ladder: every number needs a floor and a ceiling 21

5.2 The accuracy matrix, from real runs 22

5.3 Forgetting, as a curve 23

5.4 Accuracy vs forgetting vs learning 23

5.5 Is a difference real? Paired differences 24

5.6 What a “good” number is, metric by metric 24

6 Our Study: PCD in Continual Learning 25

6.1 The research question 25

6.2 How PCD maps onto continual learning 25

6.3 The four research questions 25

6.4 The fair-comparison rules 27

7 How We Know It Is Right 28

7.1 Checks that the code is PCD (the old “RO1”) 28

7.2 Checks run on 7 November 28

7.3 Problems the checks caught (and what we did) 29

8 What We Found 30

8.1 RQ1: PCD vs GEM vs ER, with the same memory 30

8.2 Finding: the margin and replay-in-the-primary do the same job 31

8.3 CIFAR-100: earlier picture and the open problem 31

8.4 RQ3: per-task margins and the schedule **32**

8.5 RQ2 and RQ4 (Salah's questions) **33**

9 The Six Questions You Must Be Able to Answer **35**

10 The Plan to 19 November **37**

Quick reference **38**

CHAPTER 1

1 · The Project in One Page

Read this first. No formulas.

The flow: catastrophic forgetting → the professor's method (PCD) → PCD applied to continual learning → four research questions → fair tests against the standard methods → an honest answer in a 12-page paper by 19 November.

The problem. A neural network trained on task A and then on task B usually forgets A. This is *catastrophic forgetting*. Continual learning (CL) is the field that tries to stop it: one model learns a sequence of tasks and should stay good at all of them.

The professor's method. Dr. AlHajri and Dara Varam published *Priority-Constrained Descent* (PCD) in TMLR 2026 [1]. PCD trains with several losses where one matters most (the *primary*) and the others are conditions it must respect (the *secondaries*). At every step it moves as close as possible to the primary's direction, but every secondary must still get at least a fraction τ of its own progress. The paper tested PCD on model compression, not on continual learning.

Our project. We apply PCD to continual learning: the new task is the primary, and each old task is a secondary that must not be pushed backwards (and, with $\tau > 0$, must even improve a little). This is project idea #3 from the course list.

What we are *not* claiming. Protecting old tasks this way already exists: GEM (2017) is exactly PCD with $\tau = 0$, and ε-SOFT-GEM (2020) already added a margin. So we do not claim a new method. Our job is a **controlled study**: which parts of PCD actually change continual-learning behaviour, in which setting, and at what cost. "It helps a little, here, for this reason" and "it does not help" are both real answers.

The four research questions (the paper's RQ1–RQ4; they follow the professor's project description).

RQ	In plain words	Owner	Status (7 November 2026)
RQ1	With a memory of old examples: does PCD's margin τ help, and does it matter what goes into the main direction?	Faris	**fair rerun done** (MNIST)
RQ2	With no memory: can PCD replace EWC's hand-tuned weight λ when nobody knows the right scale?	Salah	MNIST done: PCD-EWC $+0.9$ (task-IL, lr tuned); tie in class-IL
RQ3	Should different old tasks get different protection (by age, or by how much each is being forgotten), and should τ shrink over time? *The core of the professor's idea.*	Faris	MNIST done: tiny effect
RQ4	What should PCD's running averages do when a new task starts?	Salah	scouting (1 seed)

The old "RO1" ("is our code really PCD?") is not a research question any more. It is a check: it passed, and it gets two sentences in the paper (Chapter 7).

Where we are, in four sentences.

1. On Split MNIST, with GEM and PCD given exactly the same memory and the same learning rate, PCD **forgets 1.6–3.1 points less** than a tuned GEM, for a cost in learning new tasks; final accuracy is a tie with GEM (+0.5) and clearly above plain replay. **But when each method also picks its own learning rate, the forgetting gain disappears** ($+0.4 \pm 0.8$): a smaller learning rate is itself a way to forget less. (F-017, F-024–F-026)
2. PCD's margin and "mixing replayed old examples into the main direction" do **the same job**: with one in place, the other adds little. (F-017)
3. The professor's priority idea (RQ3) barely matters on Split MNIST: protecting the task that is slipping most lowers forgetting by about 0.4 points, and shrinking τ over time makes things slightly worse. MNIST has only 4 old tasks; CIFAR-100 (9) is the real test. (F-019, F-020)
4. The harder benchmark (CIFAR-100) now has a valid ladder at 5 epochs per task: fine-tuning 7.9%, ER 17.3%, joint 38.5%. The earlier "broken ceiling" was a 1-epoch artifact. ER is 21 points below the ceiling there (7 on MNIST), and PCD so far only ties ER, so the hard benchmark is where the real gap is. (F-021, F-015)

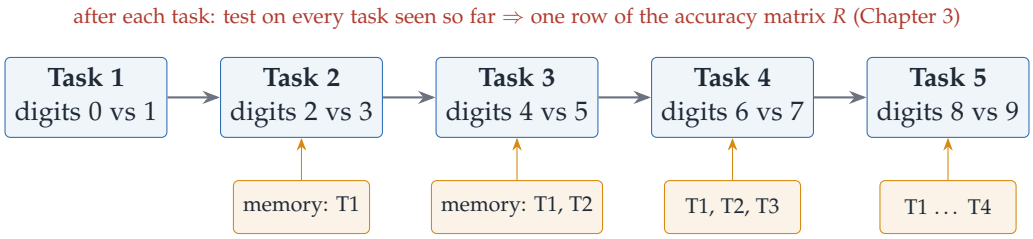
CHAPTER 2

2 · The Setup: Data, Models and the Words We Use

What the computer actually does in one run, and a dictionary for every term in this guide.

2.1 One continual-learning run, as a picture

A *run* trains one model on a fixed sequence of tasks, one after another, and tests it after each task. Here is a Split MNIST run with a replay memory:



the memory holds at most M old examples in total (200 or 500), split evenly across the old classes

Mental model While training on task 3, the model sees task 3's images in full, plus (for replay methods) a few stored images of tasks 1 and 2. It never sees the rest of tasks 1–2 again. Forgetting is what happens to tasks 1–2 while it learns task 3.

2.2 The two datasets

	Split MNIST (the fast lab)	**Split CIFAR-100** (the real test)
What the images are	handwritten digits 0–9, 28 × 28 grey pixels	small colour photos (32 × 32) of 100 kinds of object: apple, bicycle, whale, …
How it is split	5 tasks of 2 digits: {0,1}, {2,3}, …, {8,9}	10 tasks of 10 classes each
Training images	about 54,000 (after holding out 10% for validation)	about 45,000
Network	a small MLP: 2 hidden layers of 256 units	a reduced ResNet-18 (a standard small convolutional network)
Training per task	1 pass over the task's data (1 epoch)	5 epochs (to be revised, Chapter 8)
Time per run (our laptop)	about 50 seconds	about 8 minutes
Random guessing	10% (1 in 10 digits)	1% (1 in 100 classes)
Why we use it	sweep many settings cheaply; find what matters	the benchmark reviewers expect; much harder

> **Why it matters** Everything is decided on Split MNIST first because it is 10 times cheaper. Only the settings that survive are run on CIFAR-100. A finding that holds only on MNIST is a weak finding; the paper needs both.

Task orders. The digits could be paired differently ({0,7}, {6,8}, ...). A result that only holds for one particular order is luck. The final runs use 3 fixed orders (order 0 = the natural one; orders 1 and 2 are fixed random shuffles of the classes, the same for every method).

2.3 The two ways to remember

Rehearsal (with a memory)
keep a small box of old examples (M = 200 or 500 images). Use them while learning new tasks.
Methods: ER, GEM, A-GEM, ε-SOFT-GEM, **PCD** (RQ1, RQ3)

Regularization (no memory)
store no images. Remember which weights mattered for each old task, and penalize moving them.
Methods: EWC, **PCD-EWC** (RQ2)

Why have both: some applications cannot store old data (privacy, memory limits). The professor's description asks for both forms.

2.4 Dictionary: every word in this guide

Word	What it means here
task	one group of classes learned together (e.g. "digits 2 vs 3").
catastrophic forgetting	accuracy on old tasks collapses while a new task is learned.
stability / plasticity	stability = keeping old tasks; plasticity = learning new ones. Pushing one usually costs the other.
gradient	the direction in which changing the weights increases a loss fastest; training steps the opposite way.
step, learning rate (lr)	one weight update; the lr η is how far each step moves (the grid $\{0.01, 0.03, 0.1\}$ is run for every method; earlier tables use 0.1).
epoch	one full pass over a task's training images.
replay memory / buffer, M	the box of stored old examples; M is how many (200 or 500 in total).
ER (experience replay)	mix a few stored old examples into every training batch. Simple, strong baseline.
fine-tuning	just train on each new task, no protection. The *floor*.
joint training	train on all tasks' data at once (not continual). The *ceiling*.
GEM	"walls": bend each step so that no old task's loss (on its stored examples) goes up. PCD with $\tau = 0$.
GEM's γ **(memory strength)**	a knob in GEM's released code that pushes steps further toward old tasks; $\gamma = 0$ is the textbook GEM.
A-GEM, ε**-SOFT-GEM**	cheaper GEM with one averaged wall; and that with a margin. Closest prior art.
EWC	penalty $\lambda \times$(how far the important weights moved). λ must be tuned.
Fisher information F	EWC's per-weight "how much did this weight matter" score.
primary / secondary	PCD's main objective (the new task) / its conditions (the old tasks).
wall (constraint)	the condition "old task j must not lose (or must gain a τ-share of) progress"; *active* when it actually bends the step.
margin τ	how much progress each old task is guaranteed, from 0 ("don't hurt it") to 1 ("as much as if we trained on it alone").
τ_j**, rule, schedule**	RQ3: a separate margin per old task, set by a rule (oldest, newest, forgetting); a schedule shrinks τ as tasks pile up.
normalization (EMA)	PCD divides each gradient by a running average of its size so that τ means the same for every loss.
class-IL / task-IL	tested among all classes / told which task and tested among its classes only (Chapter 3).
seed	the random start (initial weights, batch order). Same seed + same settings = same result, bit for bit.
validation / test split	10% of training images held out for choosing settings / the untouched test images used only for the final numbers.
exploration / confirmatory	cheap runs that choose settings / the final pre-registered runs that go in the paper.
mean $\pm$ **std**	average over seeds $\pm$ how much the seeds disagree.
paired difference	method A minus method B *on the same seed*, then averaged. Cancels the luck of the seed.
pre-registration	the hypotheses and what would refute them, written *before* the final runs.
Mammoth	the standard CL code library we run everything in (from the DER authors [8]).
run ID, F-0xx	every run has an ID; F = Faris's machine, S = Salah's; the number is the experiment.

CHAPTER 3

3 · Background You Need

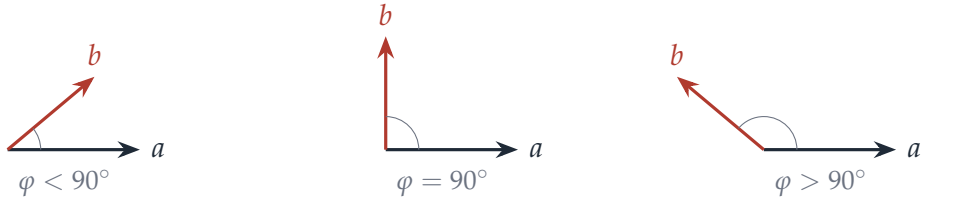
Dot products, gradient descent, and how continual learning is measured.

3.1 The dot product tells you whether two directions agree

For two vectors $a, b \in \mathbb{R}^n$,

$$a^\top b \;=\; \sum_{k=1}^{n} a_k b_k \;=\; \|a\|\,\|b\| \cos\varphi, \tag{1}$$

where $\|a\| = \sqrt{a^\top a}$ is the length of a and φ is the angle between a and b. Lengths are never negative, so the **sign** of $a^\top b$ is the sign of $\cos\varphi$:



Sign of the dot product $a^\top b > 0$ means a and b point into the same half of space. $a^\top b < 0$ means they point more than 90° apart.

Two facts we use later. If $\|a\| = \|b\| = 1$, then $a^\top b = \cos\varphi$ exactly. And the *projection* of b onto the direction of a is $\frac{a^\top b}{\|a\|^2}\, a$: the part of b that lies along a.

3.2 Gradient descent, seen through the first-order picture

Training minimizes a loss $L(\theta)$ over the parameters $\theta \in \mathbb{R}^n$. Let $g = \nabla L(\theta)$ be the gradient. If we move the parameters by a small step Δ, the first-order Taylor expansion says

$$L(\theta + \Delta) \;\approx\; L(\theta) + g^\top \Delta. \tag{2}$$

Gradient descent takes $\Delta = -\eta\, d$ with a step size $\eta > 0$ and a direction d (plain SGD uses $d = g$). Then the change in loss is

$$L(\theta - \eta d) - L(\theta) \;\approx\; -\eta\, g^\top d. \tag{3}$$

First-order descent A direction d lowers L for a small enough step if $g^\top d > 0$, and raises it if $g^\top d < 0$ (at $g^\top d = 0$ the first-order term says nothing). The bigger $g^\top d$, the more progress per unit of step. With $d = g$ we get $g^\top g = \|g\|^2 \geq 0$, which is why following the gradient works.

Careful

"Small enough" is doing real work. Equation (2) ignores curvature. If the loss is β-smooth (its gradient changes by at most β per unit of distance), a step $\eta < 2/\beta$ along $d = g \neq 0$ is guaranteed to lower it; for a general d with $g^\top d > 0$ the condition is $\eta < 2g^\top d/(\beta\|d\|^2)$. With a large step, a positive dot product promises nothing. Every "guarantee" in this guide is first-order in this sense.

3.3 Continual learning: the setting

A model sees tasks $1, 2, \ldots, T$ one after another. While training on task t it cannot see the full data of earlier tasks. Two ways to keep something from the past:

- a small **replay memory** $\mathcal{M}$ of stored examples from old tasks (*rehearsal* methods);
- a **penalty** that discourages moving parameters that mattered for old tasks (*regularization* methods, e.g. EWC).

Class-incremental vs task-incremental. In Split MNIST the 10 digits are cut into 5 tasks of 2 digits each. At test time:

- **class-incremental (class-IL)**: the model must pick the right digit out of all 10. This is hard, and it is our headline setting.
- **task-incremental (task-IL)**: the model is told which task the image comes from, so it only chooses between that task's 2 digits. This is much easier.

Our framework (Mammoth) reports both from the same run.

Why it matters A model that forgets everything except the last task gets about 2/10 of the digits right in class-IL. That is why fine-tuning sits near 19.7% on Split MNIST (F-001): the "floor" of class-IL.

3.4 How continual learning is measured: the accuracy matrix

After finishing training on task t, we test on every task i. That gives the **accuracy matrix** $R_{t,i}$: row t = "after training task t", column i = "tested on task i". All metrics come from R, and only one script computes them (`eval/evaluate.py`).

The four metrics

$$A_T = \frac{1}{T}\sum_{i=1}^{T} R_{T,i} \qquad \text{final average accuracy (higher is better)} \tag{4}$$

$$F_T = \frac{1}{T-1}\sum_{i=1}^{T-1}\Big(\max_{t\in\{i,\dots,T-1\}} R_{t,i} - R_{T,i}\Big) \qquad \text{average forgetting (lower is better)} \tag{5}$$

$$\mathrm{LA} = \frac{1}{T}\sum_{i=1}^{T} R_{i,i} \qquad \text{learning accuracy: plasticity} \tag{6}$$

$$\mathrm{BWT} = \frac{1}{T-1}\sum_{i=1}^{T-1}\big(R_{T,i} - R_{i,i}\big) \qquad \text{backward transfer (GEM's definition)} \tag{7}$$

In words: A_T is how good the final model is on everything. F_T is how far each old task fell from its best. LA is how well each task was learned when it was new. A method can lower forgetting by learning each new task less well, which is why we always look at F_T and LA together.

Worked example

Three tasks. After each task we test on the tasks seen so far:

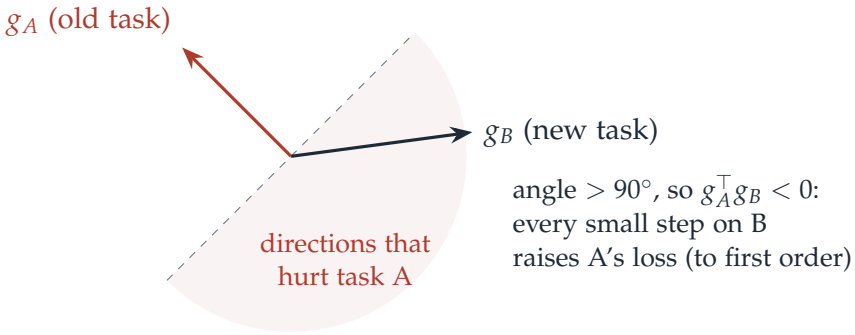
$$R = \begin{bmatrix} 95 & \cdot & \cdot \\ 70 & 96 & \cdot \\ 60 & 80 & 97 \end{bmatrix}.$$

$A_3 = (60+80+97)/3 = 79.0$. $\mathrm{LA} = (95+96+97)/3 = 96.0$. Forgetting: task 1 fell from its best 95 to 60 (35 points), task 2 from 96 to 80 (16 points), so $F_3 = (35+16)/2 = 25.5$. Here $\mathrm{BWT} = -25.5$ too, because each task's best was the moment it was learned.

3.5 Why forgetting happens, in one line

Let g_A be the old task's gradient and g_B the new task's. Training on B steps along $d = g_B$. By (3), the old task's loss changes by

$$\Delta L_A \;\approx\; -\eta\, g_A^\top g_B. \tag{8}$$



Mental model If the two gradients agree ($g_A^\top g_B > 0$), learning B also helps A. If they conflict ($g_A^\top g_B < 0$), every small step on B undoes some of A (to first order). Forgetting is a sign problem of a dot product. Every method in this guide is a way of controlling that

sign.

3.6 The existing fixes

Method	What it does	Formula (one step)
Fine-tuning	Nothing: just train on the new task. Lower reference.	$d = \nabla L_t$
Joint	Train on all tasks' data at once. Offline ceiling, not a CL method.	$d = \nabla \sum_i L_i$
ER [7]	Mix a replay batch from memory into every step.	$d = \nabla \left[L_t + L_{\mathcal{M}} \right]$
EWC [6]	Penalize moving weights that mattered for old tasks. F_k is the Fisher information (how much weight k mattered), θ^* the weights at the end of the old task.	$L_t + \frac{\lambda}{2} \sum_k F_k (\theta_k - \theta_k^*)^2$
GEM [3]	Project the new step so that no old task's memory loss rises: one constraint per old task.	$\min_d \lVert d - g \rVert^2$ s.t. $g_j^\top d \geq 0$
A-GEM [4]	Cheaper GEM: one constraint on the average memory gradient.	same, with one g_{ref}
ε-SOFT-GEM [5]	A-GEM with a positive, normalized margin ε.	one constraint, margin ε

Why it matters EWC mixes the two goals with a weight λ whose right value depends on how big the penalty is compared with the task loss. That changes with the model, the data and the number of tasks, so λ must be re-tuned. GEM's constraint $g_j^\top d \geq 0$ only forbids hurting old tasks. It does not ask them to improve. PCD addresses both.

CHAPTER 4

4 · How PCD Works

The professor's method, step by step. Inspired by the PCD project page (daravaram.github.io/PCD) and the paper.

The flow: the idea (anchor to the primary) → the QP → its picture and closed form → normalization → rescale → the threshold → many secondaries → the algorithm → what it does and does not promise.

4.1 The idea: anchor to the primary, distort the minimum

Most multi-objective methods treat every loss as an equal peer: they average the gradients, or search for weights. PCD instead says one loss is the deliverable (the *primary*, gradient g_0) and the others are conditions (the *secondaries*, gradients $g_1, \dots, g_k$).[1] The rule:

Mental model Go where the primary wants to go. Bend only as much as needed so that every secondary still gets a guaranteed share of progress. If the primary already gives every secondary its share, do not bend at all.

4.2 The PCD problem

Each step solves a small quadratic program (QP) on the *normalized* gradients $\tilde{g}_i$ (Section 4.4 explains the tilde):

$$\tilde{d}^* \;=\; \arg\min_{\tilde{d}\in\mathbb{R}^n} \tfrac{1}{2}\|\tilde{d}-\tilde{g}_0\|^2 \quad \text{s.t.} \quad \tilde{g}_j^\top \tilde{d} \;\geq\; \tau\,\|\tilde{g}_j\|^2, \qquad j=1,\dots,k. \tag{9}$$

Read it piece by piece:

- $\frac{1}{2}\|\tilde{d}-\tilde{g}_0\|^2$: stay as close as possible to the primary's direction.
- $\tilde{g}_j^\top \tilde{d}$: the first-order progress the step gives secondary j, by (3).
- $\tau\|\tilde{g}_j\|^2$: the progress secondary j would get if we followed its own gradient ($\tilde{g}_j^\top \tilde{g}_j = \|\tilde{g}_j\|^2$), times τ. So each constraint says: *give secondary j at least a fraction τ of the progress it would get on its own.*
- $\tau = 0$: the constraint becomes $\tilde{g}_j^\top \tilde{d} \geq 0$, "do not hurt j". With raw gradients that is **exactly GEM**.

One dial $\tau \in [0,1]$ is the only knob. $\tau = 0$ only forbids first-order harm to the secondaries. Larger τ demands that they actively improve. In continual learning, τ is the stability–plasticity dial.

[1] The paper numbers the primary 1 and the secondaries $2, \dots, K$. We number the primary 0, so that secondary j is past task j.

4.3 The picture, and the closed form for one constraint

Each constraint is a half-space: the set of directions on one side of a "wall" perpendicular to $\tilde{g}_j$. Solving (9) means projecting $\tilde{g}_0$ onto the region inside all the walls. With one secondary the answer has a closed form.

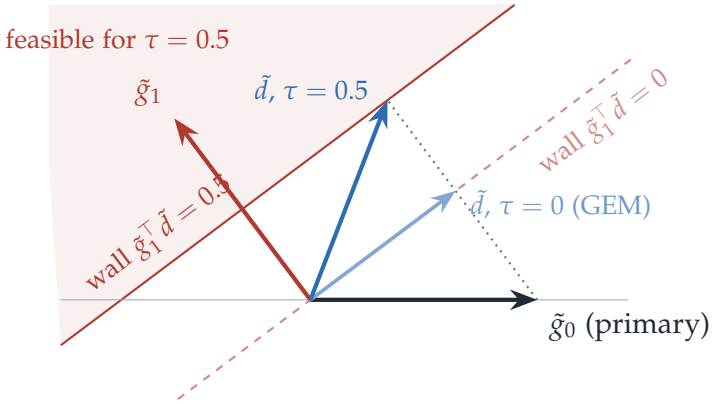


Figure 1: One old task, unit vectors, $\cos\varphi = -0.6$. The primary $\tilde{g}_0$ points into the forbidden side. PCD moves it along $\tilde{g}_1$ (dotted) until it reaches the wall. $\tau = 0$ puts the wall through the origin (GEM). $\tau = 0.5$ moves the wall out, so $\tilde{d}$ bends further toward the old task. The numbers are worked out in the example below.

Deriving it. Write the answer as $\tilde{d} = \tilde{g}_0 + \lambda\tilde{g}_1$ with $\lambda \geq 0$: we only ever add some of $\tilde{g}_1$, never subtract (this is what the KKT conditions of (9) give; Section 4.7). If $\tilde{g}_0$ already satisfies the wall, take $\lambda = 0$. Otherwise, choose λ to land exactly on the wall:

$$\tilde{g}_1^\top(\tilde{g}_0 + \lambda\tilde{g}_1) = \tau\|\tilde{g}_1\|^2 \quad \Longrightarrow \quad \lambda = \frac{\tau\|\tilde{g}_1\|^2 - \tilde{g}_1^\top\tilde{g}_0}{\|\tilde{g}_1\|^2}.$$

Both cases together:

One-constraint closed form (PCD Cor. 4.6)

$$\tilde{d} = \tilde{g}_0 + \lambda\,\tilde{g}_1, \qquad \lambda = \max\left(0,\ \frac{\tau\|\tilde{g}_1\|^2 - \tilde{g}_1^\top\tilde{g}_0}{\|\tilde{g}_1\|^2}\right). \tag{10}$$

For unit vectors, with $c = \cos\varphi = \tilde{g}_0^\top\tilde{g}_1$, this is simply

$$\lambda = \max(0,\ \tau - c). \tag{11}$$

Worked example: one wall, by hand

Take $\tilde{g}_0 = (1, 0)$ and $\tilde{g}_1 = (-0.6,\ 0.8)$. Both have length 1, and $c = \tilde{g}_0^\top\tilde{g}_1 = -0.6$: the tasks conflict.

- $\tau = 0$ **(GEM).** $\lambda = 0 - (-0.6) = 0.6$, so $\tilde{d} = (1, 0) + 0.6(-0.6, 0.8) = (0.64,\ 0.48)$.
  Check: $\tilde{g}_1^\top\tilde{d} = -0.384 + 0.384 = 0$, exactly on the wall. Primary progress $\tilde{g}_0^\top\tilde{d} = 0.64$.
- $\tau = 0.5$. $\lambda = 0.5 + 0.6 = 1.1$, so $\tilde{d} = (1 - 0.66,\ 0.88) = (0.34,\ 0.88)$.
  Check: $\tilde{g}_1^\top\tilde{d} = -0.204 + 0.704 = 0.5$. Primary progress $\tilde{g}_0^\top\tilde{d} = 0.34$.

- $\tau = 1$. $\lambda = 1.6$, $\tilde{d} = (0.04, 1.28)$. Primary progress 0.04, almost nothing left for the new task.
- **No conflict.** If instead $\tilde{g}_1 = (0.6, 0.8)$, then $c = 0.6 \geq \tau = 0.5$: the wall is already satisfied, $\lambda = 0$ and $\tilde{d} = \tilde{g}_0$. PCD does not bend.

These are the numbers in our toy figure (primary progress 0.64, 0.34, 0.04 at $\tau = 0, 0.5, 1$; `toy/results.json`).

4.4 Normalization: why τ means the same thing for every loss

If one loss is 100 times bigger than another, its gradient is 100 times longer, and "a fraction τ of progress" would mean very different things for the two. PCD therefore divides each gradient by a running estimate of its own typical length. For each objective i, at its s-th update (each objective keeps its own counter, so an old task that joins the constraints later starts at $s = 1$):

$$v_{i,s} = \beta\, v_{i,s-1} + (1-\beta)\, \|g_{i,s}\|^2, \qquad v_{i,0} = 0, \tag{12}$$

$$\hat{v}_{i,s} = \frac{v_{i,s}}{1-\beta^s}, \tag{13}$$

$$\tilde{g}_i = \frac{g_{i,s}}{\sqrt{\hat{v}_{i,s} + \varepsilon}}, \tag{14}$$

with $\beta = 0.999$ and $\varepsilon = 10^{-8}$ (the paper's defaults).

- (12) is an exponential moving average (EMA) of the squared gradient length. $\beta = 0.999$ means it remembers roughly the last $1/(1-\beta) = 1000$ steps.
- (13) is the bias correction. Because v starts at 0, early values are too small. If every $\|g\|^2$ were the same number q, then $v_s = (1-\beta^s)\, q$; dividing by $1-\beta^s$ gives back exactly q.
- (14) divides by the typical length, so in steady state $\|\tilde{g}_i\| \approx 1$ for every objective.

Asymptotic scale invariance (PCD Thm. B.9) Multiply a loss by any $c > 0$. Its gradient becomes $c\,g$, its EMA becomes $c^2\hat{v}$, and

$$\frac{c\,g}{\sqrt{c^2\hat{v} + \varepsilon}} \approx \frac{g}{\sqrt{\hat{v} + \varepsilon}} \qquad (\text{when } c^2\hat{v} \gg \varepsilon).$$

The normalized gradient does not change, so the PCD direction does not change. τ keeps its meaning whatever the raw scale of each loss.

Careful

This holds only while $c^2\hat{v} \gg \varepsilon$. When a gradient becomes tiny (of order $\sqrt{\varepsilon} = 10^{-4}$), ε dominates the denominator and the invariance breaks (PCD Rem. B.11). This matters for EWC-style penalties, whose gradient is zero at the old task's weights. Our scouting run saw exactly this (F-011, Section 8.5).

4.5 The rescale step

The QP gives a direction $\tilde{d}$ in normalized units. PCD then gives it the primary's raw length and takes a plain SGD step:

$$d = \tilde{d}\,\frac{\|g_0\|}{\|\tilde{d}\|}, \qquad \theta \leftarrow \theta - \eta\, d. \tag{15}$$

Multiplying by a positive number never changes the sign of a dot product, so d keeps the sign of every $\tilde{g}_j^\top \tilde{d}$. The margin itself is scaled by $\|g_0\|/\|\tilde{d}\|$, so for $\tau > 0$ the wall $\tilde{g}_j^\top d \geq \tau\|\tilde{g}_j\|^2$ need not hold for d: the τ-guarantee is about $\tilde{d}$ (PCD §4.7). If $\tilde{d} = 0$, PCD sets $d = 0$ (Alg. 1, line 18).

> **Careful**
>
> Plain SGD only. Adam rescales each coordinate differently, which can flip the signs PCD worked to guarantee (PCD §4.7). That is why every method in our study uses plain SGD.

4.6 The threshold: when protecting the old task stalls the new one

When the wall is active, how much progress is left for the primary? With unit vectors and (11),

$$\tilde{g}_0^\top \tilde{d} = \tilde{g}_0^\top (\tilde{g}_0 + \lambda \tilde{g}_1) = 1 + (\tau - c)\, c = 1 - c^2 + \tau c. \tag{16}$$

For conflicting tasks ($c < 0$) this falls as τ grows, and it turns negative when

$$\boxed{\tau > \frac{1 - c^2}{-c}} \qquad \text{(PCD Prop. 4.7).} \tag{17}$$

Past this threshold, protecting the old task makes the new task's loss go up (to first order). The threshold is below 1, so reachable with $\tau \in [0, 1]$, only when $1 - c^2 < -c$, that is when $c < (1 - \sqrt{5})/2 \approx -0.62$: only for strongly conflicting tasks.

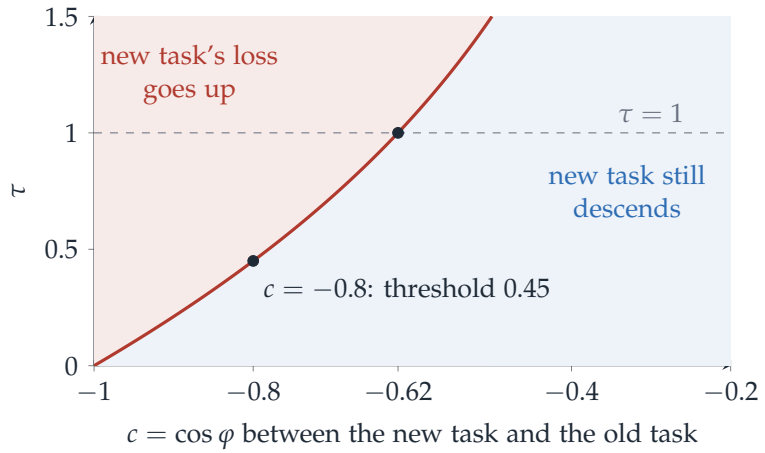


Figure 2: Prop. 4.7 for unit vectors. Below the red curve the step still lowers the new task's loss. Above it, the old task is protected at the new task's expense.

Worked example: our toy problem

In our 2-D toy (`toy/run_toy.py`) the first step has $c = -0.8006$, so the threshold is $(1 - 0.8006^2)/0.8006 = 0.4485$. The solver's primary progress at each τ:

τ	0	0.1	0.25	0.5	1
$\tilde{g}_0^\top \tilde{d}$	0.359	0.279	0.159	−0.041	−0.442

It turns negative between 0.25 and 0.5, as the threshold predicts. The QP's answer equals the hand formula (10) with maximum difference 0.0.

Why it matters In continual learning τ is a stability–plasticity trade-off. A τ that is safe when tasks conflict mildly can stall learning when they conflict strongly. We log the fraction of steps where this happens (the *primary-ascent rate*).

4.7 Many old tasks: the general solution

With k old tasks there are k walls. The KKT conditions of (9) (the optimality conditions for constrained problems) say the solution always has the form

$$\tilde{d} = \tilde{g}_0 + \sum_{j=1}^{k} \lambda_j \tilde{g}_j, \qquad \lambda_j \geq 0, \tag{18}$$

with *complementary slackness*: $\lambda_j > 0$ only if wall j is active (met with equality); a wall that is already satisfied gets $\lambda_j = 0$. The primary's coefficient is pinned to 1. The secondaries only ever add corrections.

To find the λ_j, PCD solves the smaller *dual* problem on the $k \times k$ Gram matrix $M_{ij} = \tilde{g}_i^\top \tilde{g}_j$, $i, j = 1, \ldots, k$ (not the memory size M of Chapter 4):

$$\min_{\lambda \geq 0} \ \tfrac{1}{2} \lambda^\top M \lambda - r^\top \lambda, \qquad r_j = \tau M_{jj} - \tilde{g}_j^\top \tilde{g}_0. \tag{19}$$

This is a problem in k unknowns (the number of old tasks), not in n (millions of parameters), so it is cheap.

When there is no solution. With $\tau > 0$ the walls can contradict each other: panel (c), where one old task wants $x \geq \tau$ and another $x \leq -\tau$. This happens only when the old tasks' gradients cancel, formally when 0 lies in their convex hull (PCD Thm. B.2). When there are fewer old tasks than parameters, this almost never happens (Cor. B.3). The paper's fallback is to drop the walls for that step. The alternative we study is to fall back to the $\tau = 0$ walls (GEM), which are always satisfiable by $\tilde{d} = 0$.

In our runs so far The infeasible rate was **0 in every run** (every PCD and PCD-EWC run: F-004, F-007, F-010–F-013, F-015, F-016), as Cor. B.3 predicts.

4.8 The algorithm in plain steps

One PCD step (PCD Alg. 1; our code: `src/pcd_cl/direction.py`, `qp.py`, `normalize.py`):

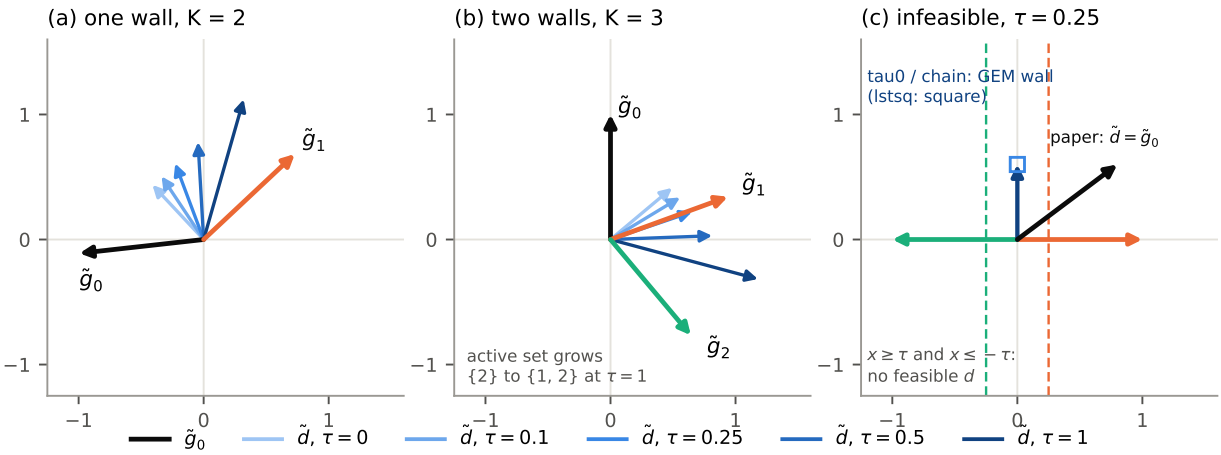


Figure 3: Our toy (`toy/run_toy.py`). (a) One wall: as τ grows, $\tilde{d}$ bends toward $\tilde{g}_1$. (b) Two walls: the set of active walls grows from $\{\tilde{g}_2\}$ to $\{\tilde{g}_1, \tilde{g}_2\}$ at $\tau = 1$. (c) Infeasible: two old tasks pointing in opposite directions, so no direction can give both positive progress.

1. Compute the raw gradients g_0 (primary) and $g_1, \ldots, g_k$ (one per old task).
2. Update each EMA (12), bias-correct (13), and normalize (14).
3. If $g_0 = 0$, set $d = 0$ (the primary-stationary halt). Otherwise, if $\tilde{g}_0$ already satisfies every wall, $\tilde{d} = \tilde{g}_0$.
4. Otherwise solve the dual (19) for λ and set $\tilde{d}$ by (18).
5. If the walls are infeasible, use the fallback.
6. If $\tilde{d} = 0$, set $d = 0$ (a fixed point, not convergence); otherwise rescale to the primary's length (15). Take a plain SGD step.

4.9 What PCD promises, and what it does not

What it gives

- When the QP is feasible, the normalized direction $\tilde{d}$ gives each secondary (its replay or penalty loss) at least a τ-fraction of first-order progress; the rescaled d keeps the sign of that progress.
- When $\tilde{g}_0$ already satisfies every wall, the step is exactly the primary's direction.
- τ has a scale-free meaning, thanks to normalization (while ε is negligible).
- One bounded dial instead of a search over loss weights.

What it does not give (we must never claim these)

- It does **not** guarantee "no forgetting". The walls are first-order and hold for small enough steps only.
- In CL the walls are computed on a few replay samples per old task. Satisfying them on those samples is not a guarantee on the old task's true loss.

- The guarantees are about the direction $\tilde{d}$ at one step, not a convergence theorem for the whole training run.
- Adam can break the sign guarantees, so we use plain SGD.

CHAPTER 5

5 · Reading a Result

How to tell whether a number is good: the ladder, the accuracy matrix, forgetting curves, and "is the difference real?"

5.1 The ladder: every number needs a floor and a ceiling

A number like "89.5%" means nothing on its own. We always place it between two references from *our own* runs:

- the **floor** = fine-tuning (no protection at all), and
- the **ceiling** = joint training (all data at once, no forgetting possible).

Between them sit the standard methods. If our method ever scored above the ceiling, or the ladder came out in the wrong order, we would know something was broken.

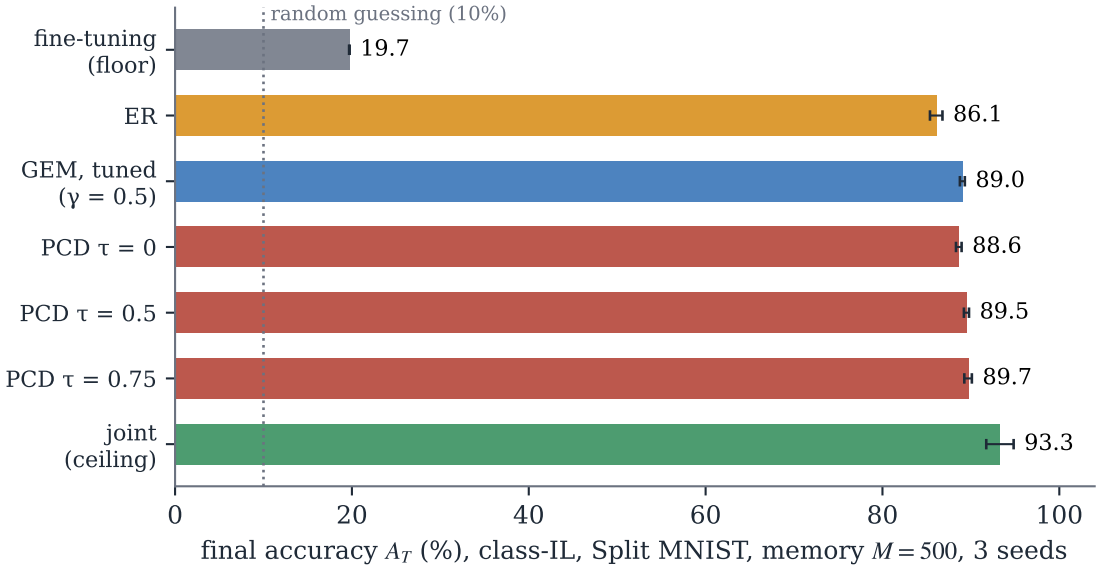


Figure 4: The ladder on Split MNIST ($M = 500$, class-IL, 3 seeds; F-001, F-002, F-017). Error bars are ± one standard deviation over seeds.

How to read the ladder The useful range is 19.7 → 93.3. Plain replay (ER) already climbs to 86.1, so the real contest is over the last 7 points. Tuned GEM closes the gap to the ceiling to 4.3 points; PCD with $\tau = 0.75$ to 3.6. In this field a consistent 0.5–1 point at the top of the ladder is small but real; what matters is whether it survives the fair checks of Chapter 7 and the harder dataset.

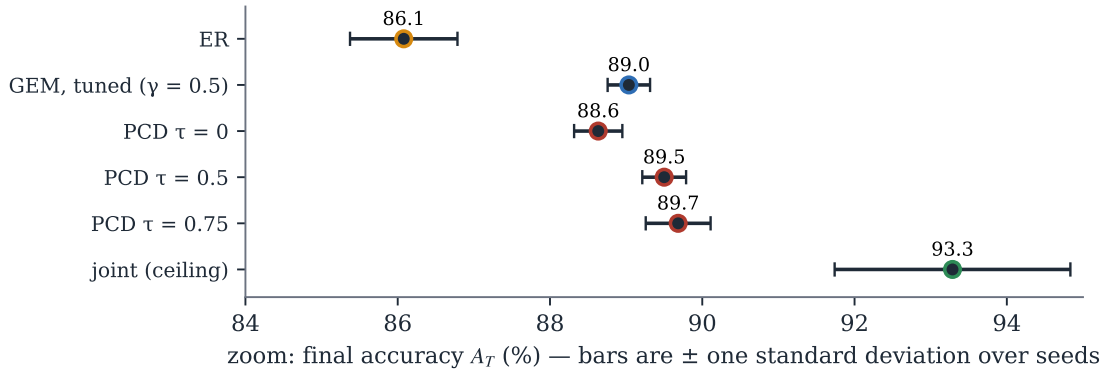


Figure 5: The top of the ladder, zoomed in. Overlapping error bars mean "we cannot tell these apart from three seeds alone", which is why we use paired differences (Section 5.5).

5.2 The accuracy matrix, from real runs

Chapter 3 defined $R_{t,i}$ = accuracy on task i after training task t. Here are three real ones (seed 0):

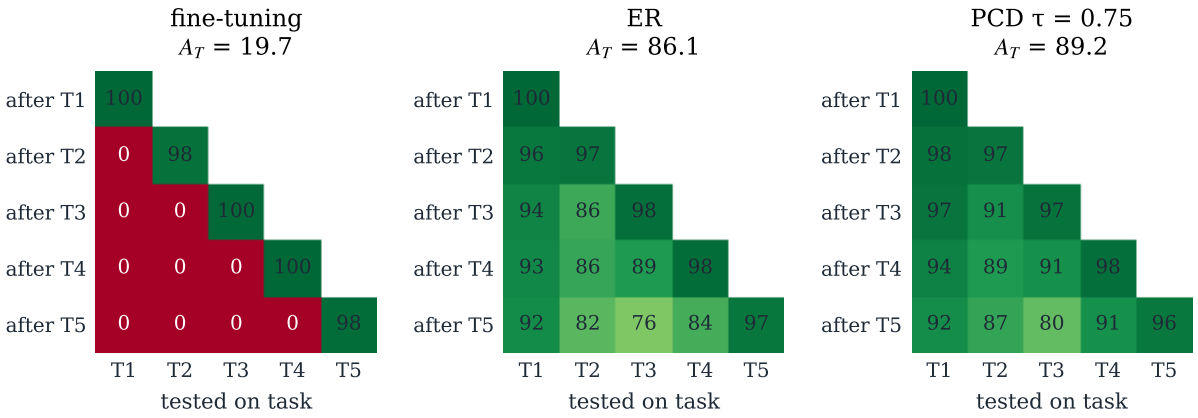


Figure 6: Real accuracy matrices on Split MNIST (seed 0; F-001, F-002, F-017). Row = after which task; column = tested on which task. Green = remembered, red = forgotten.

Reading the three matrices

Fine-tuning: the diagonal is near 100 (each task is learned perfectly when it is new), but everything below the diagonal is 0: each new task erases all earlier ones. The last row is $(0, 0, 0, 0, 98)$, so $A_T = 98/5 \approx 19.7$. That is where the class-IL floor comes from.
ER: the memory keeps old tasks alive, but they erode: task 3 falls from 98 to 76.
PCD: the same pattern, with less erosion: task 3 ends at 80, task 2 at 87.
A_T = the mean of the last row. F_T = for each old task, its best value in its column minus its last value, averaged. LA = the mean of the diagonal.

5.3 Forgetting, as a curve

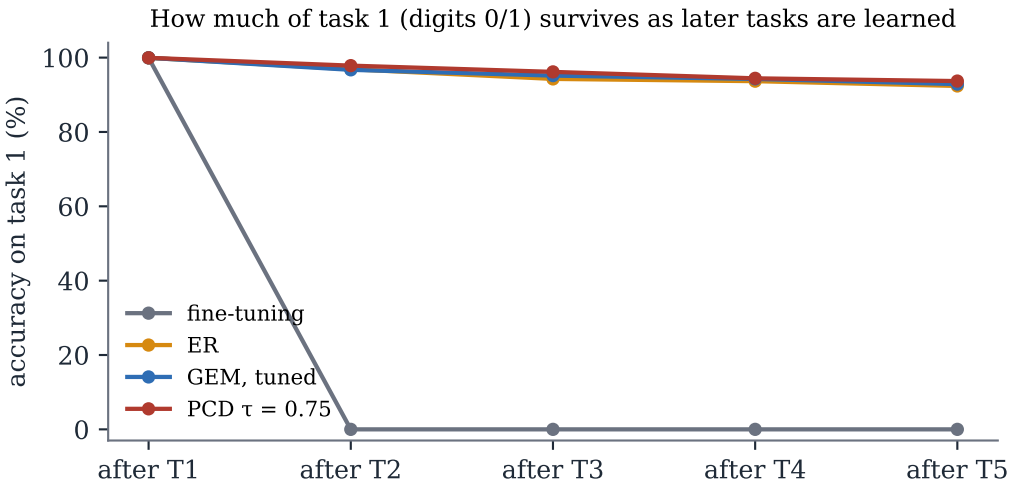


Figure 7: Accuracy on task 1 (digits 0 vs 1) as later tasks are learned; mean of 3 seeds (first column of R). Fine-tuning loses it entirely the moment task 2 starts. The three protected methods keep 92–94%.

5.4 Accuracy vs forgetting vs learning

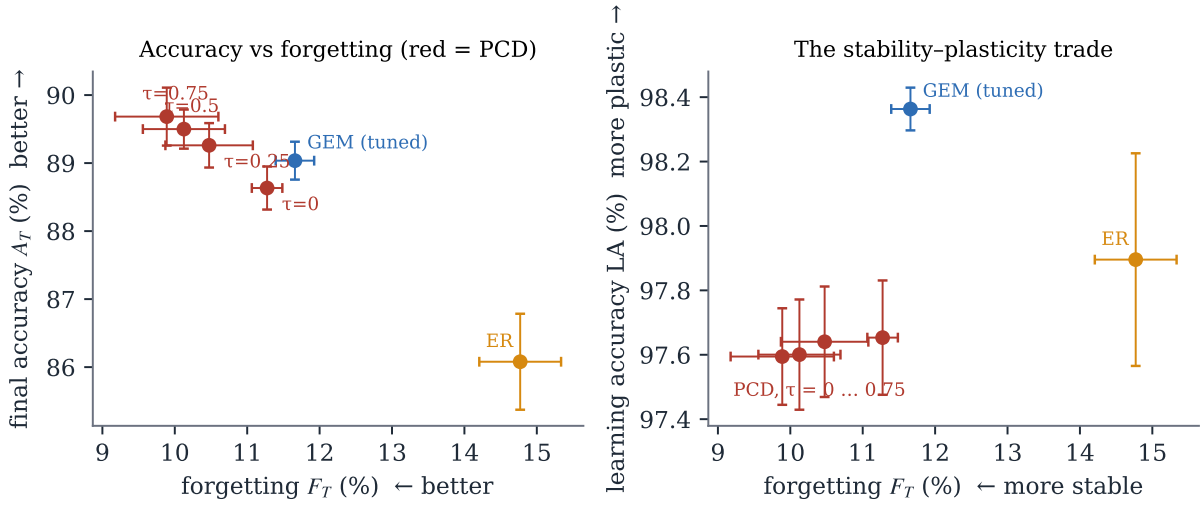


Figure 8: Left: final accuracy against forgetting (up and to the left is better). Right: learning accuracy against forgetting, the stability–plasticity trade (F-002, F-017; 3 seeds).

Mental model A method can "forget less" simply by refusing to learn new tasks well. So we never read F_T alone: we look at it together with LA. In the right panel, PCD moves left (more stable) and slightly down (0.8 points less plastic) compared with GEM. That is a trade, not a free win; the left panel shows the trade comes out slightly positive overall.

5.5 Is a difference real? Paired differences

Each method is run with seeds 0, 1, 2. Different seeds give slightly different results, and some seeds are "easy" for every method. So instead of comparing two averages, we subtract *on the same seed*:

$$\Delta = \frac{1}{3}\sum_{s=0}^{2}\left(\mathrm{PCD}_s - \mathrm{GEM}_s\right), \qquad \text{reported as } \Delta \pm \mathrm{std}_s(\mathrm{PCD}_s - \mathrm{GEM}_s).$$

Our rule (pre-registered) A difference counts only if it is larger than one standard deviation of the paired differences, in the final runs over 3 seeds × 3 task orders, on both datasets. Smaller than that, we call it a tie.

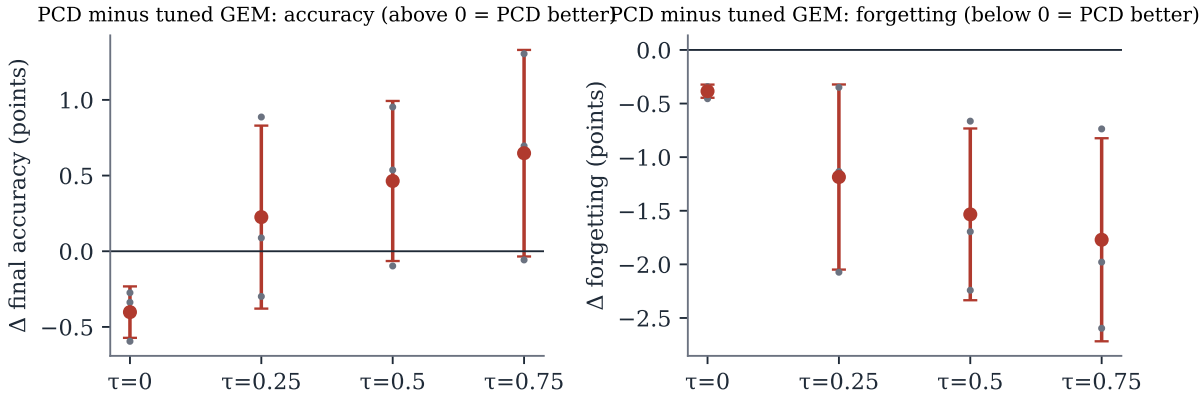


Figure 9: PCD minus tuned GEM ($\gamma = 0.5$), same seeds, Split MNIST $M = 500$ (F-017 vs F-002). Red = mean ± std; grey dots = the individual seeds.

Reading Figure 9

For accuracy (left), the error bars cross zero for every τ: a tie. For forgetting (right), $\tau = 0.5$ and 0.75 sit clearly below zero, and every seed is below zero: PCD forgets less, for real. At $\tau = 0$ PCD and GEM are almost the same, as they should be (they are nearly the same algorithm).

5.6 What a "good" number is, metric by metric

Metric	Bad	Typical (our runs, MNIST $M = 500$)	Good
A_T (class-IL) ↑	near 19.7 (the floor)	ER 86.1, GEM 89.0	close to joint, 93.3
F_T ↓	near 99 (forgets everything)	ER 14.8, GEM 11.7	near 0
LA ↑	well below fine-tuning's 99.0 (the method refuses to learn)	97.6–98.4	98–99
TIL A_T ↑	near 50 (coin flip between 2 digits)	98.5–98.9	99+ (task-IL is easy)
active-wall rate	0 (PCD does nothing) or always 1	0.50 at $\tau = 0$ to 0.91 at $\tau = 0.75$ (F-017)	informative, not a score
infeasible rate	$> 0.1\%$ of steps	0 in every run	0

CHAPTER 6

6 · Our Study: PCD in Continual Learning

The research question, the two forms, and the four research questions with what would prove each one wrong.

6.1 The research question

Research question PCD's constraint is already known in continual learning: with $\tau = 0$ it is GEM. So the question is not *whether* constraint projection works in CL. It is: **which parts of PCD change continual-learning behaviour, in which setting, and at what cost?**

Novelty boundary: what we never write

That constraining against past-task gradients is new (GEM, A-GEM); that a positive margin is new (ε-SOFT-GEM, and GEM's own γ); that PCD "guarantees no forgetting". PCD is the professor's method; we apply and study it. A clean negative result is a valid outcome.

6.2 How PCD maps onto continual learning

While training on task t, the primary is the current task and there is one secondary (one wall) per old task $j < t$:

	Rehearsal form (RQ1, RQ3)	**Regularization form** (RQ2)
Memory	a replay buffer $\mathcal{M}_j$ per old task	none
Primary g_0	$\nabla[L_t + L_{\mathcal{M}}]$: current batch plus a replay batch (default), or ∇L_t only	∇L_t
Wall g_j	$\nabla L_j(\mathcal{M}_j)$: the loss on *all* stored examples of task j (the same memory GEM uses)	$\nabla\Omega_j = c\,F_j \odot (\theta - \theta_j^*)$, the gradient of task j's EWC penalty
Closest baseline	GEM (walls, $\tau = 0$); ER (no walls)	EWC ($L_t + \lambda \sum_j \Omega_j$)

Why it matters The project description defines the main direction from "the current task together with an aggregate representation of previously acquired knowledge", as in FedPCD [2]. That is why replay goes into the primary by default, and why "what is in the primary" turned out to be one of our main findings (Chapter 8).

6.3 The four research questions

RQ1 — The margin and the primary (rehearsal form; Faris).

H1 With the same memory and number of steps, PCD with a moderate $\tau \in \{0.1, 0.25, 0.5\}$ forgets less (F_T) than GEM, while losing at most 2 points of learning accuracy.

Refuted if, on both datasets, PCD beats GEM by more than one paired standard deviation (3 seeds $\times$ 3 orders) on neither F_T nor A_T.

RQ2 — No memory: PCD-EWC vs EWC (regularization form; Salah). EWC's weight λ only means something relative to the size of the penalty. If the penalty is multiplied by an unknown scale c, the right λ changes by $1/c$. PCD normalizes each wall, so its τ should not care.

H2 (a) With an equal small tuning grid (3 values each) and $c \in \{0.01, 0.1, 1, 10, 100\}$ unknown, PCD-EWC reaches a higher task-IL accuracy than EWC on average over c. (b) Fully tuned, PCD-EWC matches or beats EWC.

Why task-IL: without a memory, both EWC and PCD-EWC sit at the 19.7% class-IL floor (they cannot tell tasks apart), so class-IL cannot distinguish them. The pre-registration named class-IL; we report both and say so in the paper (decision D-024).

RQ3 — A priority hierarchy among old tasks (Faris; the core of the professor's idea). Until now every old task got the same τ. RQ3 gives each old task its own margin τ_j, always with the same *average*, so a rule can only win by protecting the *right* task:

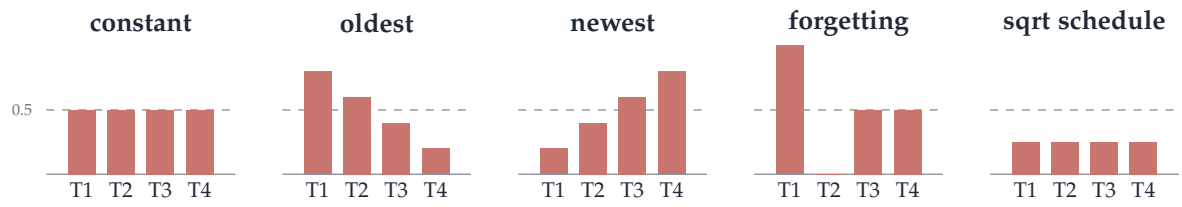


Illustration with 4 old tasks and average margin $\tau = 0.5$ (values from `src/pcd_cl/tau_policy.py`). *Forgetting* here assumes task 1 has slipped twice as much as tasks 3–4 and task 2 not at all; task 2 then gets $\tau_2 = 0$, which still means "do not hurt it". The sqrt schedule is $0.5/\sqrt{4} = 0.25$.

H4 (a) At the same average margin, the *forgetting* rule forgets less than one shared τ, at most 2 points of learning cost. (b) The shrinking schedule $\tau_0/\sqrt{k}$ learns new tasks better (higher LA), at no more than 1 point of extra forgetting.

Honest expectation: Split MNIST has at most 4 old tasks, so a hierarchy has little room; CIFAR-100 (up to 9 old tasks) is where it can matter.

RQ4 — Task boundaries (Salah). At a task switch the primary's gradient size jumps, and PCD's running average (EMA, $\beta = 0.999$) takes about 10^3 steps to catch up. Three modes: `carry` (keep every average), `reset` (restart all), `bank` (keep one per old task, restart the primary's; like FedPCD's per-client history).

H3 `reset` or `bank` lowers the post-switch normalization error and lowers F_T compared with `carry`.

6.4 The fair-comparison rules

- plain SGD (no momentum, no Adam: Adam can break the sign guarantees), batch 32, the same network, initialization and epochs for every method;
- the same memory: M counts every stored example, and PCD's walls use the same stored examples as GEM's;
- all choices made on the 10% validation split, never on the test set; every baseline gets the same tuning effort (the learning-rate grid $\{0.01, 0.03, 0.1\}$ is run for every method);
- final runs: 3 seeds $\times$ 3 task orders on both datasets; the refutation criteria are frozen *before* they run (`docs/PREREGISTRATION.md`).

CHAPTER 7

7 · How We Know It Is Right

Every check we run, and the problems the checks caught.

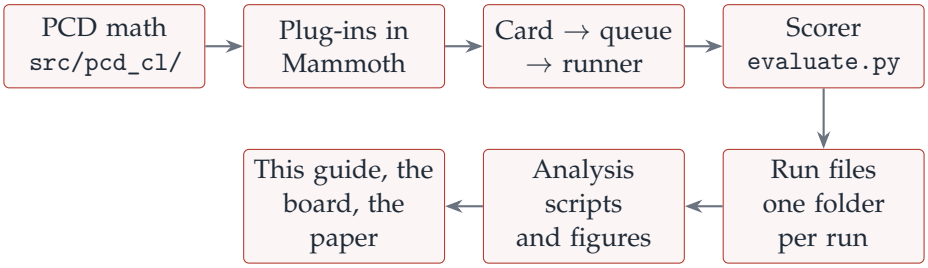


Figure 10: The pipeline. Nobody types numbers: everything flows from the run files.

7.1 Checks that the code is PCD (the old "RO1")

- 302 automated tests pass: closed-form answers, the optimality (KKT) conditions, scale invariance, and the per-task τ_j rules.
- Our solver gives the same answer as the official PCD code on all 9 test cases.
- One step of PCD with $\tau = 0$ equals one step of Mammoth's GEM to 10^{-5}.

7.2 Checks run on 7 November

Check	Result	What it proves
Rerun Mammoth's GEM, same seed, twice	identical to 0.01 points on every task	runs are deterministic: a difference between two settings is never luck of the machine
PCD in its exact-GEM setting vs Mammoth's GEM, 3 seeds (F-018 vs F-005)	$+0.9 \pm 1.2$; within 0.05–0.6 per seed once GEM's tiny 10^{-3} stabilizer is removed	our PCD really is GEM at $\tau = 0$; the left-over is rounding compounding over ∼1,400 steps
Do task orders really reorder? (order 1 vs the raw MNIST files)	all 5 tasks hold the expected digit pairs (image counts within 2, mean pixel within 0.05)	"3 task orders" will not silently be the same order
Memory-matched rerun (F-017 vs the earlier runs)	PCD's walls now use the same stored examples as GEM; results moved by 0.0–0.2 points (0.8 in one setting)	the earlier GEM comparison was not distorted by the memory difference
The ladder is in order	floor $<$ ER $<$ GEM $\lesssim$ PCD $<$ ceiling on MNIST	nothing is "too good to be true"

7.3 Problems the checks caught (and what we did)

1. **GEM saw more memory than PCD.** Mammoth's GEM builds its walls from the whole buffer; ours used 32 examples per task. Fixed: PCD's walls now use the whole buffer too. It changed almost nothing, but now the comparison is fair by construction.
2. **Mammoth's task-order option is broken** at our pinned version (it stores the order but never applies it). Fixed in our launcher; verified against the raw data.
3. **The summary script could merge different settings.** It ignored the wall memory, the τ rule and the learning rate. Fixed; the old report still reproduces row for row.
4. **New jobs fell back to a test-only backend** and crashed. No numbers were produced; fixed and re-run.
5. **The CIFAR-100 "ceiling" was below ER.** Cause: Mammoth's joint model trains once at the end, for one task's worth of epochs, so at 1 epoch per task it got 1 epoch in total. At 5 epochs per task joint reaches 38.5%, well above ER (17.3%): solved (F-021, Chapter 8).
6. **CIFAR-100 runs were not bit-for-bit repeatable** (solved). The same ER command, same seed, gave 16.6% on 5 November and 17.3% on 7 November (F-015 vs F-021); the GPU's convolution routines were choosing non-deterministic methods. Fix: a switch that forces deterministic GPU math. Test: the same CIFAR command twice gives identical accuracy matrices with the switch and up to 8 points apart per cell without it; four old MNIST runs rerun with the switch are identical to the stored ones, so no MNIST result changes (D-022).
7. **EWC crashes at large λ** (understood). Seven EWC runs stopped with a NaN loss. Not a bug: plain SGD on EWC's penalty overshoots once $\mathrm{lr} \cdot 2\lambda F_{\max} > 2$, and we measured exactly that before the crash (2.6). We wrote down what the learning-rate grid must show if this is the reason, before running it.
8. **Mammoth's A-GEM ignores the memory size** (it stores one batch per task). Excluded until a matched version exists.

CHAPTER 8

8 · What We Found

Every number: validation split, exploration. Tables say whether lr is fixed at 0.1 or tuned per method.

8.1 RQ1: PCD vs GEM vs ER, with the same memory

Split MNIST, $M = 500$, 3 seeds	$A_T \uparrow$	$F_T \downarrow$	LA↑	gap to ceiling
fine-tuning (floor) F-001	19.7	99.1	99.0	−73.6
ER F-002	86.1 ± 0.7	14.8	97.9	−7.2
GEM, tuned $\gamma = 0.5$ F-002	89.0 ± 0.3	11.7	98.4	−4.3
PCD $\tau = 0$ F-017	88.6 ± 0.3	11.3	97.7	−4.7
PCD $\tau = 0.25$ F-017	89.3 ± 0.3	10.5	97.6	−4.0
PCD $\tau = 0.5$ F-017	89.5 ± 0.3	10.1	97.6	−3.8
PCD $\tau = 0.75$ F-017	**89.7 ± 0.4**	**9.9**	97.6	−3.6
joint (ceiling) F-001	93.3	—	—	0

Paired vs tuned GEM ($\gamma = 0.5$), same seeds	ΔA_T	ΔF_T	ΔLA
PCD $\tau = 0$	-0.4 ± 0.2	-0.4 ± 0.1	−0.7
PCD $\tau = 0.25$	$+0.2 \pm 0.6$	-1.2 ± 0.9	−0.7
PCD $\tau = 0.5$ (inside H1's range)	$+0.5 \pm 0.5$	$\mathbf{-1.5 \pm 0.8}$	−0.8
PCD $\tau = 0.75$	$+0.6 \pm 0.7$	$\mathbf{-1.8 \pm 0.9}$	−0.8
PCD $\tau = 0.75$ vs *ER*	$\mathbf{+3.6 \pm 0.9}$	$\mathbf{-4.9 \pm 0.8}$	−0.3

The learning rate tuned too (F-024–F-026 at lr 0.01, 0.03; lr 0.1 rows reproduce the table above). Each method picks τ or γ by mean A_T; paired over seeds:

lr	PCD pick	GEM pick	ΔA_T	ΔF_T	ΔLA
0.01 (shared)	$\tau = 0.25$	$\gamma = 1$	$+0.5 \pm 0.2$	-3.1 ± 0.1	−2.7
0.03 (shared)	$\tau = 0.75$	$\gamma = 1$	$+0.4 \pm 0.3$	-2.2 ± 0.3	−1.3
0.1 (shared)	$\tau = 0.75$	$\gamma = 1$	$+0.5 \pm 0.8$	-1.6 ± 1.1	−0.8
own lr each	$\tau = 0.75$, lr 0.1	$\gamma = 1$, lr 0.03	$+0.5 \pm 0.5$	$+0.4 \pm 0.8$	+0.8

RQ1 on Split MNIST, after equal tuning At any shared learning rate PCD forgets 1.6–3.1 points less than tuned GEM and ties it on accuracy. But H1 says each method is tuned for accuracy, and once the learning rate is part of the tuning, GEM picks lr 0.03, where it forgets less by itself, and the forgetting gain is gone ($+0.4 \pm 0.8$; accuracy $+0.5 \pm 0.5$). **So H1 is not supported on Split MNIST.** Post hoc, not pre-registered: PCD at lr 0.03 (A_T 89.6, F_T 7.3) beats GEM's best point (89.2, 9.5) on both at once, so PCD reaches better accuracy–forgetting trade-offs; that is a frontier statement and will be reported as one. CIFAR-100 is still open.

8.2 Finding: the margin and replay-in-the-primary do the same job

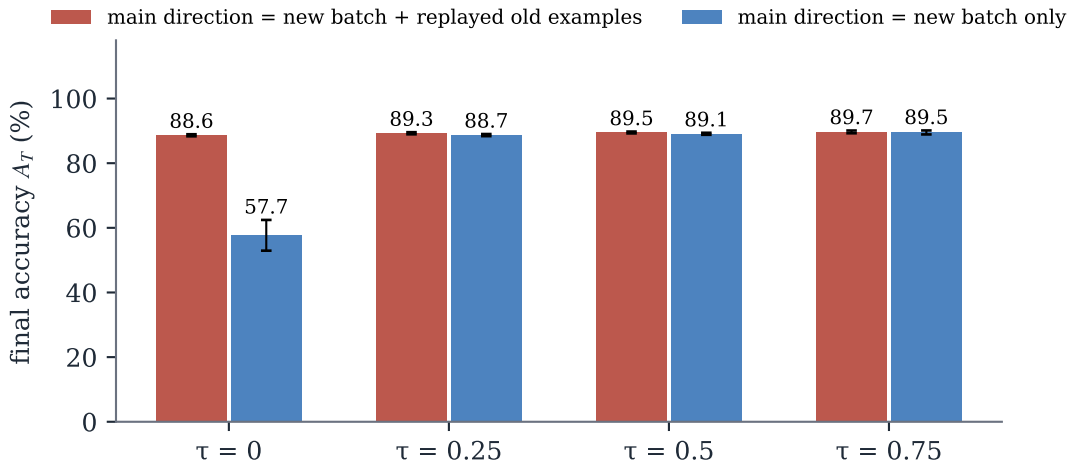


Figure 11: PCD on Split MNIST, $M = 500$, memory-matched walls, 3 seeds (F-017). Red: the main direction includes replayed old examples. Blue: the new batch only.

The most interesting result so far With the new batch alone as the main direction, $\tau = 0$ (plain GEM) collapses to 57.7%, and the margin is worth +32 points ($\tau = 0.75$: 89.5%). With replay already in the main direction, the margin adds only about +1. So "τ helps" is only true relative to what is in the primary: the margin and replay-in-the-primary buy the same stability.

Mental model Replayed old examples in the primary already pull every step toward the old tasks, so the walls have little left to correct (at $\tau = 0$ they are active on 50% of steps, F-017). Take replay out, and the walls are the only thing protecting old tasks (active on 71%), so how hard they push (τ) matters a lot. GEM's γ knob does the same: $\gamma = 0$ gives 56.8%, $\gamma = 0.5$ gives 89.0%.

8.3 CIFAR-100: earlier picture and the open problem

At 5 epochs per task, with replay in the primary, PCD equals ER (17.9 ± 0.9 vs 17.5 ± 0.9; paired $+0.4 \pm 0.6$, F-015). With the new batch only, $\tau = 0$ gives 8.4% and $\tau = 0.5$ gives 14.3% (F-016). But at 1 epoch the joint "ceiling" scored 9.4%, below ER's 13.2% (F-006), so we cannot yet place CIFAR numbers on a ladder.

CIFAR-100 status

Solved: the ladder is in order at 5 epochs per task (F-021, seed 0, validation split):

CIFAR-100, 10 tasks, 5 ep/task, $M = 500$	lr	A_T	LA	F_T
random guessing	—	1.0	—	—
fine-tuning (floor)	0.1 / 0.03	7.9 / 7.8	67.2 / 68.6	65.8 / 67.6
ER	0.1 / 0.03	17.3 / 16.5	72.5 / 69.0	61.4 / 58.3
joint (ceiling)	0.1 / 0.03	**38.5** / 36.0	—	—

The 9.4% "ceiling" was a 1-epoch artifact: Mammoth's joint trains once at the end for one

task's worth of epochs, so it had 1 epoch in total. Note how much room CIFAR leaves: ER is 21 points below the ceiling (7 on MNIST). lr 0.1 is slightly better than 0.03 for all three. One seed only. These runs were not bit-repeatable (same command: 16.6 vs 17.3, F-015 vs F-021); from F-029 on every CIFAR run uses deterministic GPU math and repeats exactly. The deterministic rerun of this ladder agrees: fine-tuning 8.1, ER 17.8, joint 38.9 (F-029, seed 0).

8.4 RQ3: per-task margins and the schedule

All runs: Split MNIST, $M = 500$, walls from every stored example, 3 seeds; each rule is compared on the same seed with the constant rule at the same average margin and primary (F-019, F-020 vs F-017). Validity: the realized average margin is exactly $\bar{\tau}$ in every rule run (0.75 for $\bar{\tau} = 0.75$), the rules really spread the margins (max−min ≈ 0.43), and the sqrt schedule's realized average (0.522 at $\tau_0 = 0.75$) is what $\tau_0 \cdot \text{mean}(1, \frac{1}{\sqrt{2}}, \frac{1}{\sqrt{3}}, \frac{1}{2})$ predicts. No infeasible steps.

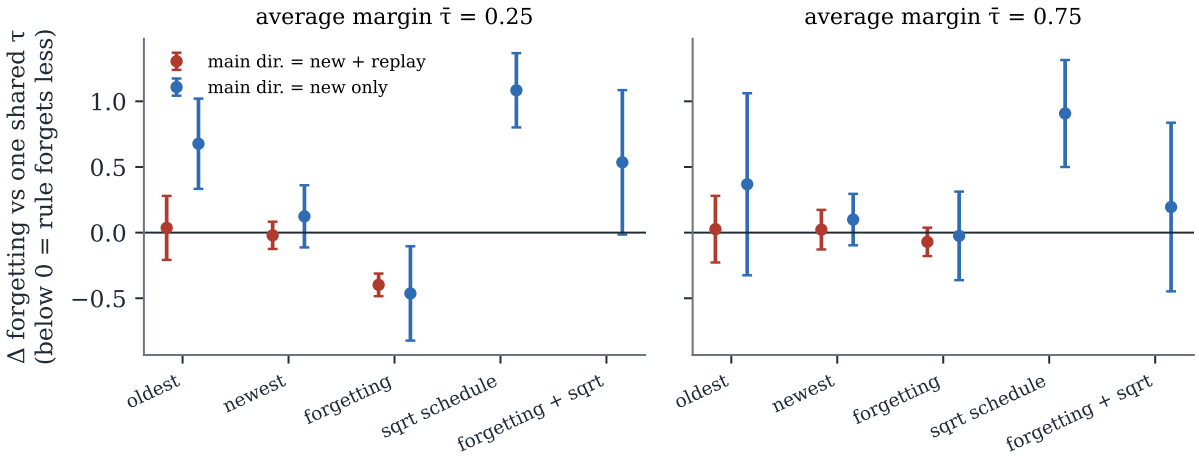


Figure 12: Change in forgetting when the margin is split by a rule instead of shared, same average margin, same seed (F-019, F-020 vs F-017). Dots = mean, bars = ± std over 3 seeds.

Arm (paired vs one shared τ)	$\bar{\tau}$, primary	ΔA_T	ΔF_T	ΔLA
forgetting rule	0.25, new + replay	$+0.3 \pm 0.1$	$\mathbf{-0.4 \pm 0.1}$	-0.0
forgetting rule	0.25, new only	$+0.4 \pm 0.3$	$\mathbf{-0.5 \pm 0.4}$	-0.0
forgetting rule	0.75, either	≈ 0	≈ 0	≈ 0
oldest-first rule	0.25, new only	-0.5 ± 0.3	$+0.7 \pm 0.3$	$+0.0$
newest-first rule	any	within ± 0.1	within ± 0.2	0.0
sqrt schedule	0.25, new only	-0.8 ± 0.2	$+1.1 \pm 0.3$	$+0.0$
sqrt schedule	0.75, new only	-0.6 ± 0.4	$+0.9 \pm 0.4$	$+0.1$

RQ3 on Split MNIST (a) Protecting the task that is slipping most (the *forgetting* rule) is the only rule that helps, and only a little: about 0.4–0.5 points less forgetting at $\bar{\tau} = 0.25$, consistent on every seed with a replay primary; at $\bar{\tau} = 0.75$ the effect disappears. Protecting the oldest task first is slightly harmful. (b) The shrinking schedule does the opposite of what H4b predicted: it forgets about 1 point *more* and does not learn new tasks any better (ΔLA ≤ 0.1).

Mental model Why so small: Split MNIST has at most 4 old tasks, learning accuracy is already 97.6–98.4%, so there is no plasticity left to buy back, and with $\bar{\tau} = 0.75$ almost every wall is already active, so redistributing the margin changes little. The schedule just lowers the average margin late in training, which is "less protection", so more forgetting. The test that matters is CIFAR-100: 9 old tasks and learning accuracy around 70%, where both effects have room. That run is planned once the CIFAR setup is fixed.

8.5 RQ2 and RQ4 (Salah's questions)

All runs: Split MNIST, no memory, 3 seeds, validation split. EWC: F-022 (lr 0.1), F-027 (lr 0.01, 0.03); PCD-EWC: F-023 (lr 0.1), F-028 (lr 0.01, 0.03). The penalty is multiplied by an unknown scale $c \in \{0.01, 0.1, 1, 10, 100\}$. Multiplying EWC's penalty by c is *exactly* the same as using $\lambda \cdot c$, so EWC(λ, c) is the EWC run at $\lambda \cdot c$. Each method gets the same grid: 3 settings (EWC $\lambda \in \{10, 100, 1000\}$; PCD-EWC $\tau \in \{0.1, 0.5, 1\}$) at lr 0.1 as pre-registered, then the same 3 settings at each lr in $\{0.01, 0.03, 0.1\}$ (9 points each). For each c, a method keeps its best point by the mean over seeds; a point counts only if all 3 seeds finished.

Validity checks. (1) Class-IL: every EWC and PCD-EWC run sits at the floor (19.4–19.7%, only the last task survives), as expected without memory, so RQ2 is read in task-IL; the pre-registration named class-IL, and the switch is disclosed (D-024). (2) EWC runs crash (the loss becomes NaN) once $\lambda c \geq 10^4$ at every lr, and sometimes at $\lambda c = 10^3$ (1 of 3 seeds at lr 0.1, 2 of 3 at lr 0.03, none at lr 0.01). This is not a bug: plain SGD on EWC's quadratic penalty is only stable while lr $\cdot\, 2\lambda F_{\max}$ stays below about 2; we measured that number in crashed and finished runs and it separates them (checks report §2, §4; our prediction of *which* runs would crash at lower lr was partly wrong, because the importance F grows as lr falls). (3) PCD-EWC: 0 crashes, 0 infeasible steps, 135 of 135 runs finished. (4) The picks below were recomputed by a second script that reads the run folders directly, and a third time by the figure code: all three agree.

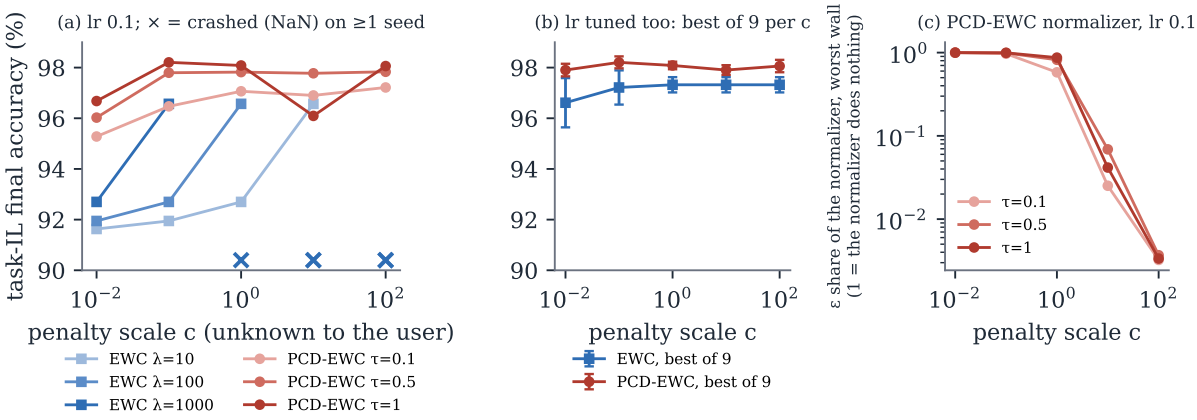


Figure 13: Task-IL final accuracy against the unknown penalty scale c (mean of 3 seeds). (a) lr 0.1 only, every setting (F-022, F-023): EWC's best λ moves with c and its useful settings run into crashes. (b) The learning rate tuned too: each method's best of 9 points per c, bars = seed std (F-022, F-027, F-023, F-028). EWC recovers to 96.6–97.3%; PCD-EWC stays at 97.9–98.2%. (c) How much of PCD-EWC's normalizer is the safety constant ε (worst wall, mean over steps, lr 0.1). At $c \leq 1$ the walls are so small that ε dominates and the normalizer does almost nothing.

	lr 0.1 only (3 points each)		lr tuned too (9 points each)		
c	EWC	PCD-EWC (Δ)	EWC pick: task-IL	PCD-EWC pick: task-IL	paired Δ
0.01	92.7 ± 3.4	$96.7\ (+4.0 \pm 3.5)$	λ1000, lr .03: 96.6 ± 1.0	τ1, lr .03: 97.9 ± 0.2	$+1.3 \pm 1.0$
0.1	96.6 ± 0.8	$98.2\ (+1.6 \pm 0.8)$	λ1000, lr .03: 97.2 ± 0.7	τ1, lr .1: 98.2 ± 0.2	$+1.0 \pm 0.8$
1	96.6 ± 0.8	$98.1\ (+1.5 \pm 0.9)$	λ1000, lr .01: 97.3 ± 0.3	τ1, lr .1: 98.1 ± 0.1	$+0.8 \pm 0.4$
10	96.6 ± 0.8	$97.8\ (+1.2 \pm 1.5)$	λ100, lr .01: 97.3 ± 0.3	τ0.1, lr .03: 97.9 ± 0.2	$+0.6 \pm 0.4$
100	**all crash**	98.1	λ10, lr .01: 97.3 ± 0.3	τ1, lr .1: 98.1 ± 0.2	$+0.7 \pm 0.5$

Usable EWC points out of 9 with the lr grid: 9, 9, 7, 4 and 1 for $c = 0.01 \ldots 100$ (21 of the 27 EWC runs at $c = 100$ crash). Averaged over c, paired over seeds: lr 0.1 only, $+2.1 \pm 1.7$ (per seed $+2.6$, $+3.4$, $+0.2$; 4 values of c); lr tuned, $+0.9 \pm 0.6$ (per seed $+1.4$, $+1.0$, $+0.2$; all 5 values of c). Class-IL, both views: -0.1 (a tie at the floor).

RQ2 on Split MNIST: the answer after equal tuning (a) Once the learning rate is tuned too, PCD-EWC is still ahead of EWC in task-IL at every penalty scale, but by 0.6–1.3 points instead of 1.2–4.0. Averaged over c: $+0.9 \pm 0.6$, above the pre-registered bar of one paired standard deviation; every one of the 15 (seed, c) pairs is positive, but seed 2 shows only $+0.2$. In class-IL, the pre-registered metric, the two are tied at the floor, so H2a as written (class-IL) is not supported; in task-IL it is, narrowly. (b) Most of the lr-0.1 gap was EWC being run at the wrong learning rate: at lr 0.01 EWC is stable and good at every c it can reach. (c) What remains is mostly a *level* difference, not robustness: EWC's own best varies only 0.7 points across c (96.6–97.3), PCD-EWC's 0.3 (97.9–98.2). EWC's single best setting (97.9 at $\lambda c = 1000$, lr 0.1) crashes on one seed in three and is therefore not usable.

Mental model Why EWC breaks at lr 0.1: the size of EWC's step on the penalty is $\text{lr} \cdot \lambda c$ times the importance F. Make λc large and the step overshoots; it overshoots more each time, and the weights blow up. A smaller learning rate pushes that edge further out, so with lr in the grid EWC can almost always find a setting that trains. PCD-EWC never takes a step proportional to the penalty: it asks the QP for a direction with "at least a fraction τ of each wall's own progress", and the length is set by the new task, so it never crashes. Why PCD-EWC dips at $c = 0.01$ and lr 0.1: the normalizer divides by $\sqrt{\hat{v} + \varepsilon}$; when the walls are tiny, ε wins (panel c) and the margin protects less. At lr 0.03 that dip is gone (97.3–97.9 for every τ).
What we can say in the paper: with equal tuning, PCD-EWC matches or beats EWC by about 1 point in task-IL at every penalty scale and never diverges, while EWC needs the learning rate as an extra knob to avoid divergence at large scales. We cannot say PCD-EWC is "much more robust to the penalty scale": with lr tuned, EWC is nearly flat too. Split MNIST only; one dataset, 3 seeds, validation split.

RQ4 scouting (F-010, 1 seed). A_T: `bank` 84.9, `reset` 84.8, `carry` 84.2. But the normalization error is *lower* for `carry` (0.59) than for `reset` (0.63) or `bank` (0.64): the proposed mechanism is not supported so far. Infeasible steps: 0 in every run, so the fallback question will be reported as "does not arise".

CHAPTER 9

9 · The Six Questions You Must Be Able to Answer

If you can say these out loud in your own words, you understand the project. Model answers below; cover them and try first.

1. What is forgetting, and how do we measure it?

Model answer

Forgetting is the drop in an old task's accuracy while new tasks are learned. After each task we test on all tasks so far, which fills the accuracy matrix R. Three numbers come from it: final accuracy A_T (mean of the last row: how good the final model is on everything), forgetting F_T (how far each old task fell from its best, averaged), and learning accuracy LA (mean of the diagonal: how well each task was learned when new). We read F_T together with LA, because a method can forget less just by learning less.

2. Why do we compare against fine-tuning, ER, GEM and joint?

Model answer

Fine-tuning is the floor (no protection: 19.7% on MNIST class-IL). Joint is the ceiling (all data at once: 93.3%). ER is the simplest strong method (store examples, mix them in), so anything fancy must beat it. GEM is PCD's closest relative: it is exactly PCD with $\tau = 0$, so "PCD vs GEM" isolates what PCD's extra parts add.

3. What does τ do, and why is $\tau = 0$ GEM?

Model answer

Each old task is a wall on the step. $\tau = 0$ means "the step must not increase this old task's loss", which is exactly GEM's condition. $\tau > 0$ means "the step must also decrease it, by at least a fraction τ of what training on it alone would give". PCD also normalizes each gradient so τ means the same for every loss, and it keeps the step as close as possible to the new task's direction.

4. What have we found, and why does it make sense?

Model answer

(i) With the same memory and the same learning rate, PCD forgets 1.6–3.1 points less than tuned GEM on Split MNIST, at a cost in learning; overall accuracy is a tie, and it clearly beats ER. That makes sense: a positive margin actively repairs old tasks, and that repair takes a bit of the step away from the new task. But a smaller learning rate also lowers forgetting, so when each method tunes its own learning rate the gain disappears: on MNIST, H1 is not supported. (ii) The margin and replay-in-the-primary do the same job: if replayed old examples are already in the main direction, the walls rarely bind and

τ adds about 1 point; without them, τ is worth 32 points. (iii) Splitting the margin by a rule (RQ3) barely matters on MNIST: the forgetting rule helps by about 0.4 points, and the shrinking schedule hurts, because with only 4 old tasks and learning accuracy already near 98% there is little left to redistribute or buy back. (iv) On CIFAR-100, PCD ties ER so far, while ER sits 21 points below the ceiling; that is where both the hard test of RQ1 and the room for RQ3 are. (v) Without memory (RQ2), when nobody knows the penalty's scale, PCD-EWC beats EWC in task-IL by about 2 points at lr 0.1, but by only about 1 point once the learning rate is tuned too. Most of the lr-0.1 gap was EWC blowing up: its step grows with the penalty, and a smaller learning rate pushes that edge away. PCD-EWC never blows up, because its step does not grow with the penalty. With the learning rate tuned, EWC is nearly flat across scales too, so the honest claim is "about 1 point better and never diverges", not "much more robust".

5. Why is RQ3 the core of the professor's idea?

Model answer

His title is *Hierarchical Optimization of the Stability–Plasticity Dilemma*. His description asks for a hierarchy among past tasks built from recency or measured forgetting, and for a tolerance that may be annealed. Plain PCD treats every old task the same. RQ3 tests exactly his proposal: give each old task its own τ_j (by age or by how much it is slipping), at the same average, and shrink τ as tasks pile up.

6. What can we *not* claim?

Model answer

Not that walls against old-task gradients are new (GEM, A-GEM), not that a margin is new (ε-SOFT-GEM, GEM's γ), not that PCD guarantees no forgetting (the walls are first-order, hold for small steps, and are checked only on stored examples, not the true old-task data), and not that the results hold beyond what we ran (two datasets, plain SGD, one network each). Our claim is a controlled study of which PCD parts matter in CL.

CHAPTER 10

10 · The Plan to 19 November

Who does what, day by day. The laptop runs overnight on the nights marked (night).

Day	Work	Exit check
Sat 7	fair RQ1, RQ3 (MNIST), CIFAR ladder diagnosis, RQ2 at lr 0.1, deterministic CIFAR (done); night: learning-rate grid for every method on MNIST, RQ1 + RQ3 on CIFAR-100 (3 seeds)	RQ1, RQ2 tables cross-checked; CIFAR repeats exactly
Sun 8	read the night's results: lr chosen per method, RQ2 under equal tuning, CIFAR RQ1 + RQ3	lr chosen on validation for every method
Mon 9	Salah: RQ4 runs and review of PR #7; both: freeze the protocol and the pre-registration	both sign off on the freeze
Tue 10 – Fri 13	final runs: MNIST 3 seeds × 3 orders (hours); CIFAR-100 best settings + baselines (night, about 14 h over two nights)	every final table cell filled from a run ID
Sat 14 – Sun 16	analysis, figures, results and discussion sections; cut to 12 pages	reviewer pass: every number has a run ID
Mon 17	full draft; optional: send to Dara for a quick read	draft complete, nothing "pending"
Tue 18	fixes from feedback, final polish	builds with no warnings
Wed 19	**submit**	

Risks

(1) CIFAR-100 runs take 3–4 minutes (fine-tuning, ER, joint) to about 18 minutes (GEM, PCD) each; the final grid must fit in two nights, so only the best settings go to CIFAR. (2) RQ2–RQ4 need Salah's machine time and review. (3) Effects are small, so the paper must be precise, not hyped.

Quick reference

Symbol	Meaning
g_0, $\tilde{g}_0$	primary gradient (the current task), raw and normalized
g_j, $\tilde{g}_j$	secondary gradient for old task j, raw and normalized
$\tilde{d}$, d	PCD direction before and after the rescale
τ, τ_j	the margin; RQ3: one per old task, same average
λ_j	how much of $\tilde{g}_j$ is added to the primary; > 0 only when wall j is active
$c = \cos\varphi$	cosine between primary and an old task; $c < 0$ means they conflict
β, ε	EMA memory (0.999) and stability constant (10^{-8})
M	replay memory size (number of stored examples)
$R_{t,i}$	accuracy on task i after training task t
A_T, F_T, LA	final accuracy, forgetting, learning accuracy
class-IL / task-IL	pick among all classes / among the given task's classes

Result	Formula
First-order change	$L(\theta - \eta d) - L(\theta) \approx -\eta\, g^\top d$
Forgetting	$\Delta L_A \approx -\eta\, g_A^\top g_B$
PCD problem	$\min_{\tilde{d}} \frac{1}{2}\|\tilde{d} - \tilde{g}_0\|^2$ s.t. $\tilde{g}_j^\top \tilde{d} \geq \tau_j \|\tilde{g}_j\|^2$
One wall	$\lambda = \max\big(0, (\tau\|\tilde{g}_1\|^2 - \tilde{g}_1^\top \tilde{g}_0)/\|\tilde{g}_1\|^2\big)$; unit vectors: $\lambda = \max(0, \tau - c)$
Normalization	$v \leftarrow \beta v + (1-\beta)\|g\|^2$, $\hat{v} = v/(1-\beta^s)$, $\tilde{g} = g/\sqrt{\hat{v} + \varepsilon}$
Rescale	$d = \tilde{d}\,\|g_0\|/\|\tilde{d}\|$ ($d = 0$ if $\tilde{d} = 0$)
Paired difference	$\Delta = \frac{1}{S}\sum_s (\mathrm{A}_s - \mathrm{B}_s)$, counted only if $\|\Delta\| > \mathrm{std}_s$

Where to look in the repository. `docs/RESEARCH_PLAN.md` (the plan) · `docs/PREREGISTRATION.md` (what would refute us) · `docs/DECISIONS.md` (why things are the way they are) · `docs/reports/` (generated tables) · `paper/draft/` (the paper) · `literature/pdfs/` (the 12 papers, with a reading guide).

References

[1] D. Varam and M. I. AlHajri. Not all objectives are born equal: Priority-constrained descent for hierarchical multi-objective optimization. *Transactions on Machine Learning Research*, 2026.

[2] Y. Irshaid, D. Varam and M. I. AlHajri. FedPCD: Priority-constrained federated aggregation. arXiv:2610.13508, 2026.

[3] D. Lopez-Paz and M. Ranzato. Gradient episodic memory for continual learning. *NIPS*, 2017.

[4] A. Chaudhry, M. Ranzato, M. Rohrbach and M. Elhoseiny. Efficient lifelong learning with A-GEM. *ICLR*, 2019.

[5] G. Hu, W. Zhang, H. Ding and W. Zhu. Gradient episodic memory with a soft constraint for continual learning. arXiv:2011.07801, 2020.

[6] J. Kirkpatrick et al. Overcoming catastrophic forgetting in neural networks. *PNAS* 114(13), 2017.

[7] A. Chaudhry et al. On tiny episodic memories in continual learning. arXiv:1902.10486, 2019.

[8] P. Buzzega, M. Boschini, A. Porrello, D. Abati and S. Calderara. Dark experience for general continual learning: a strong, simple baseline. *NeurIPS*, 2020.